



GPU



Computación Paralela

- División de tareas de computo
- Puede encontrarse en diferentes niveles

- ☐ Pipeline
- ☐ Threads
- ☐ Multi-Cores
- ☐ APIs (Ej MPI - Message Passing Interface)
- ☐ Clusters - Grids

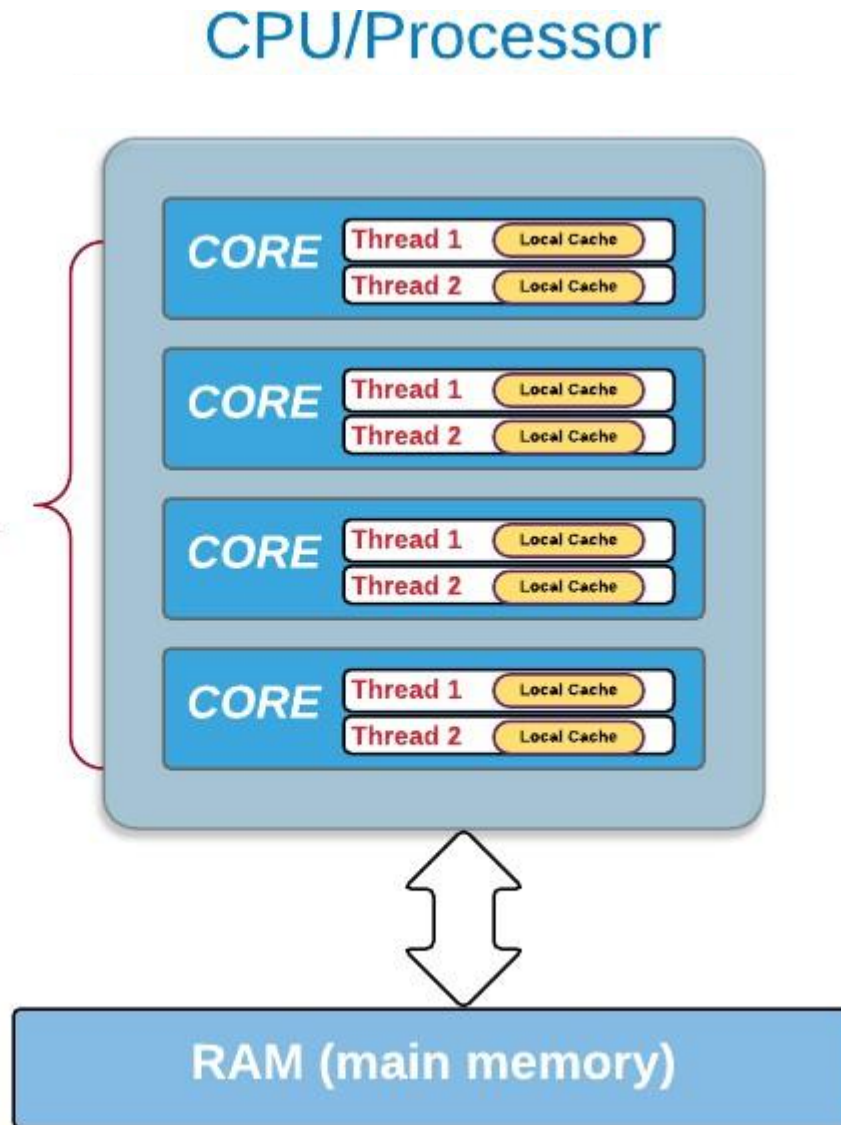


¿Que es un hilo/thread ?

- Parte de un programa que se puede ejecutar de manera independiente de otras partes de código.
- Se implementan junto al Sistema Operativo con una librería
- Un proceso puede tener por ejemplo 2 hilos.
- Corren dentro del mismo núcleo (core)
- Al compartir recursos (ej. Memoria) se debe cuidar la sincronización (ej. Exclusión tipo Mutex)

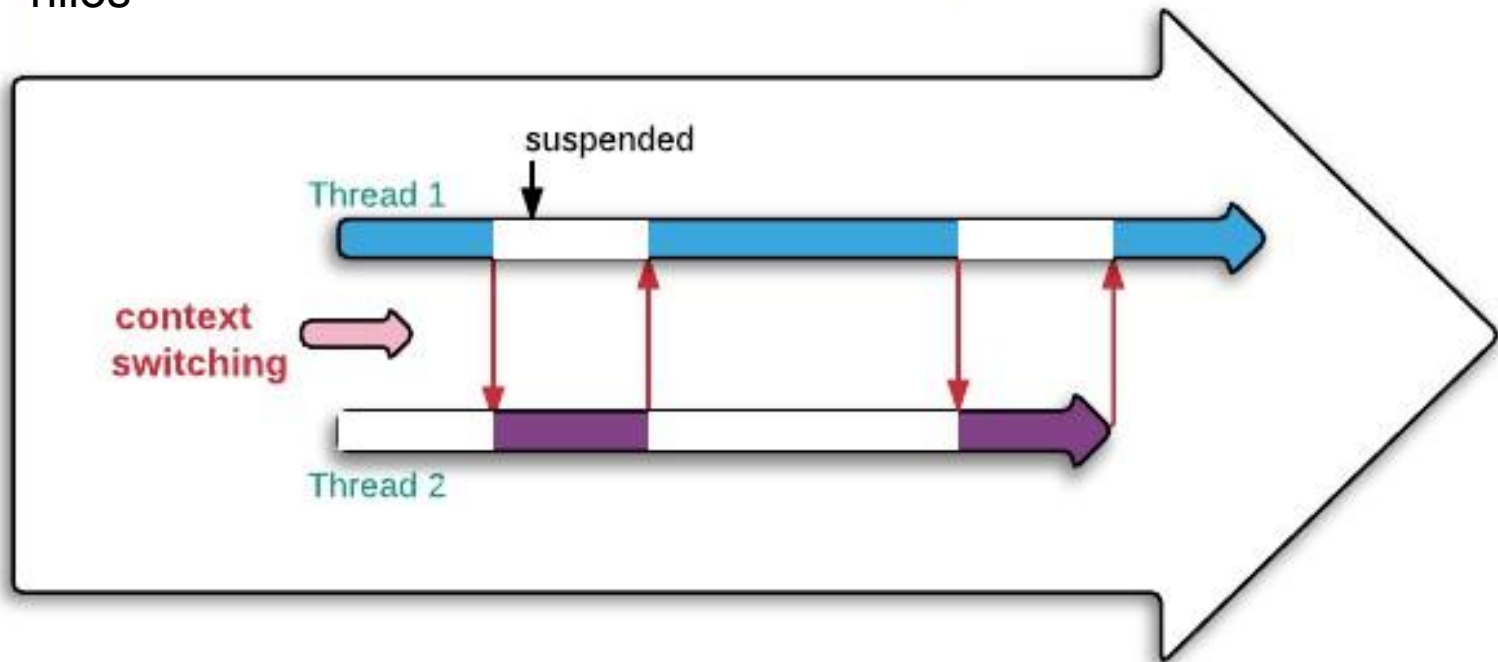
¿Que es un hilo/thread ?

Si el CPU soporta Hyper-Threading, por ejemplo podemos tener dos hilos por core, sino es un solo hilo por core.



¿Que es un hilo/thread ?

Ejemplo de un proceso corriendo en un core con dos hilos



Aparece el concepto de Mutex (Mutual Exclusion) para poder usar mismos recursos (ej. dato en RAM) por ambos hilos

Ejemplo Intel

- Procesador Intel® Core™ i9-10900K
- Cantidad de núcleos : 10
- Cantidad de subprocesos: 20

Tecnología Intel® Hyper-Threading ‡



La Tecnología Intel® Hyper-Threading ofrece dos cadenas de procesamiento por núcleo físico. Las aplicaciones con muchos subprocesos pueden realizar más trabajo en paralelo, completando antes las tareas.

 Encuentre productos con Tecnología Intel® Hyper-Threading ‡



GPU

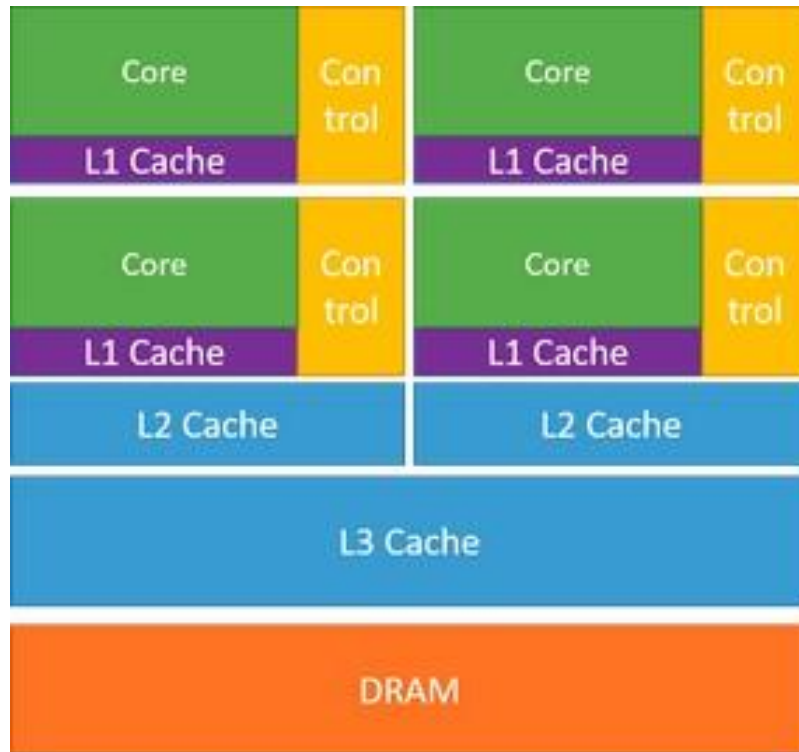
- Diseñada para ejecutar miles de threads en paralelo
- **Menor velocidad de ejecución de UN thread que una CPU**
- Pero más cantidad.



Optimización

- En CPU
- Se busca optimizar código **secuencial**
- En GPU
- Se busca optimizar código en **paralelo**
- GPU tiene 5 ó 10 veces más de bus a la memoria

GPU



CPU



GPU



Fabricantes de GPU

- NVIDIA (tomaremos esta marca de referencia)
- AMD
- ATI
-



Nvidia - CUDA

- Nace en 2006 para facilitar la programación de las GPU
- Es una Arquitectura de Software y Hardware
- CUDA Architecture + CUDA Programming Model
- Es un toolkit para programar, extensión de C
- A partir de CUDA 6, se puede acceder tanto a memoria RAM como la memoria de la GPU



CUDA

- Creado en 2007
- Se programa en C, C++, OpenCL (como ejemplos)
- Es un pipeline de **operaciones de punto flotante.**

Paralelización

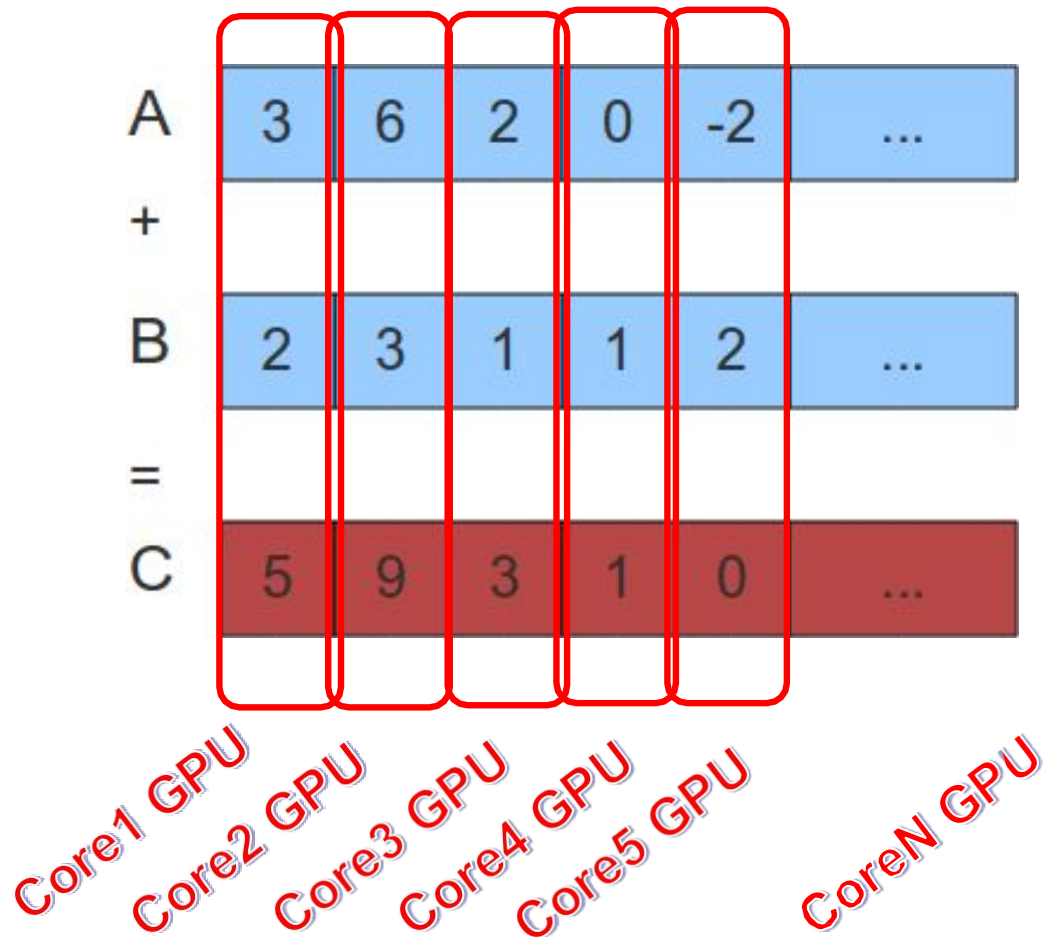
- Ejemplo de suma de dos vectores
- En CPU de un hilo, recorre cada elemento, sumo e incrementa, de manera secuencial.
- En CPU de dos hilos de podría paralelizar, sumar elementos a la vez

$$\begin{array}{r} \text{A} \\ + \\ \text{B} \\ = \\ \text{C} \end{array} \begin{array}{|c|c|c|c|c|c|} \hline 3 & 6 & 2 & 0 & -2 & \dots \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|} \hline 2 & 3 & 1 & 1 & 2 & \dots \\ \hline \end{array} \begin{array}{|c|c|c|c|c|c|} \hline 5 & 9 & 3 & 1 & 0 & \dots \\ \hline \end{array}$$

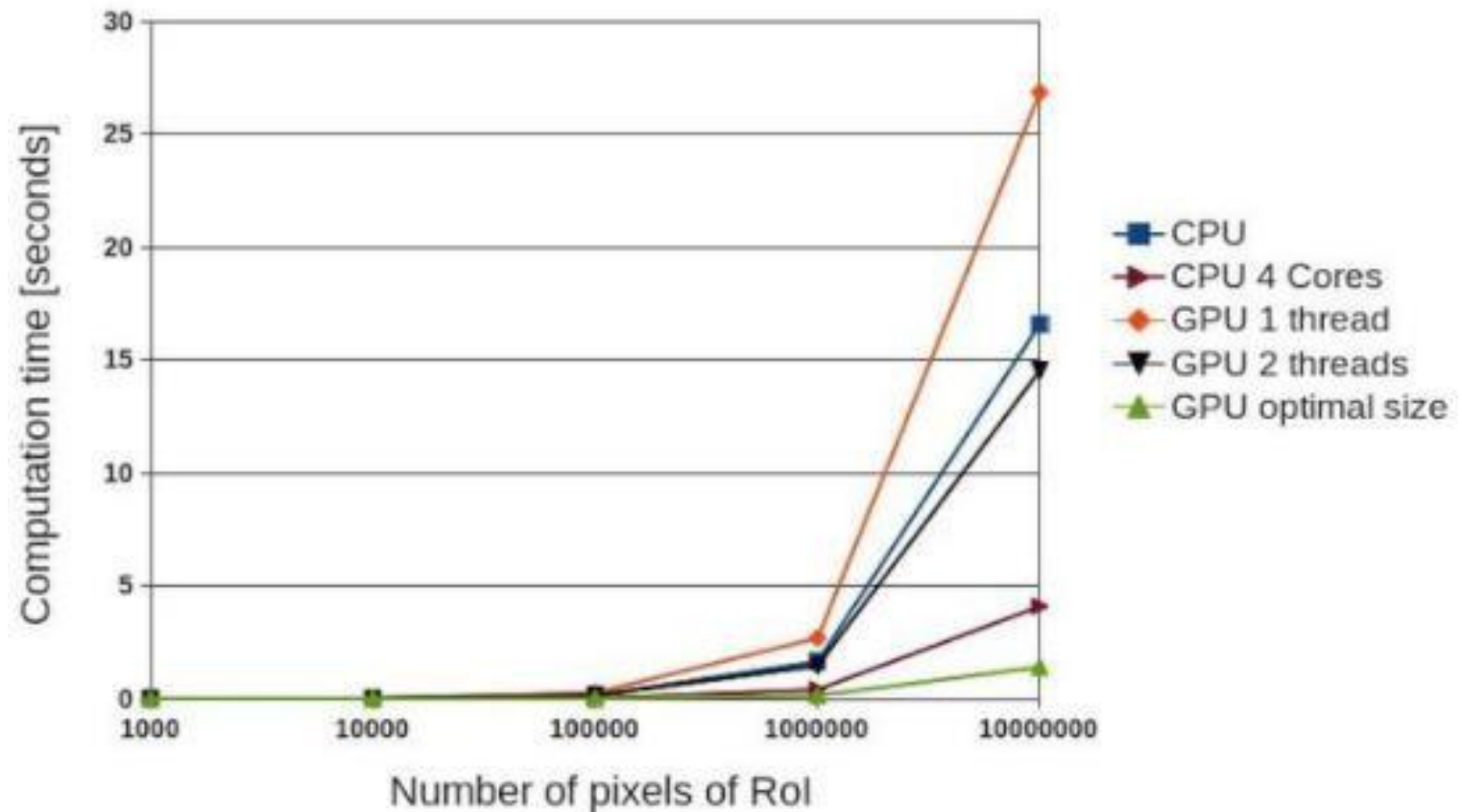
Y con la GPU ?

Paralelización - GPU

- Ejemplo de suma de dos vectores
- Cada suma podría ser ejecutada simultáneamente por un procesador (virtual) de la GPU



Paralelización – GPU vs CPU



Ejemplo de tiempo de respuesta en alteración de pixeles. Observar el rendimiento de un hilo de GPU

Pasos para ejecutar código

Definiciones

“Host”: La CPU del
“Device”: sistema. La GPU

Para ejecutar un programa de CUDA

- 1) Se copia los datos de entrada de host-memory a device-memory
- 2) Se carga el programa en device-memory y se ejecuta
- 3) La salida del programa se copia de device-memory a host-memory

Pasos para ejecutar código

Definiciones

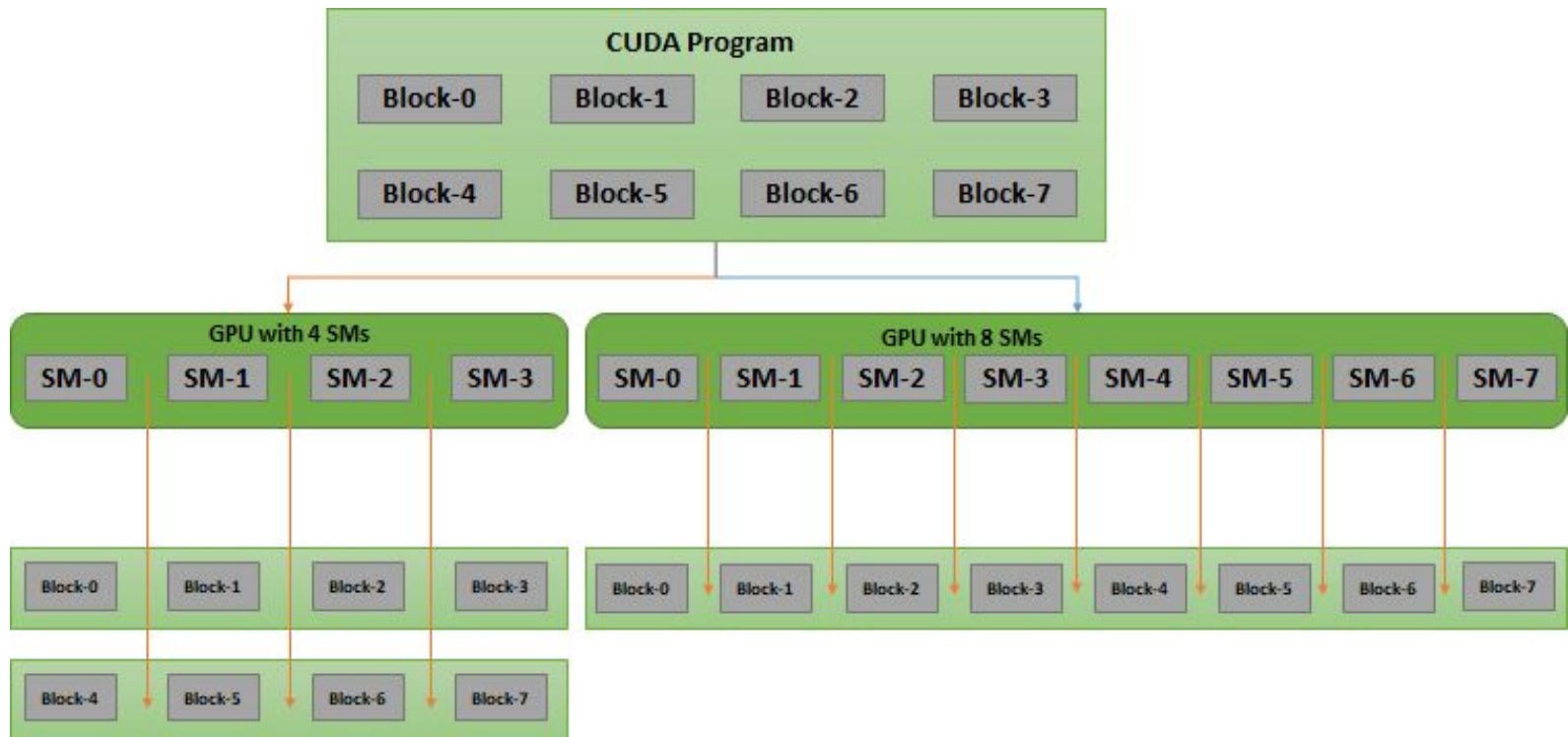
“Block”:	Grupos de hilos
“Grid”:	Grupo de Blocks

“SM (Stream Multiprocessor)”

Puede correr N Blocks

Cada GPU contiene diferentes cantidades de SM

Pasos para ejecutar código

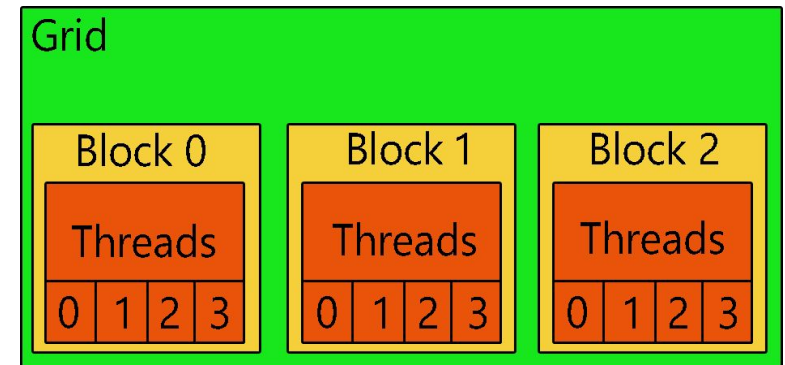


Un programa CUDA al correr en una GPU con 4 SM ó en una GPU con 8 SM. Es transparente para el programador. Lo soluciona CUDA.

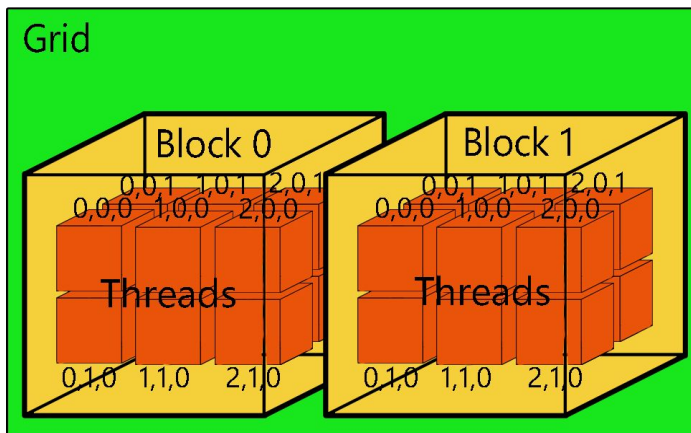
Grids y Blocks

- Bloques con hilos de una, dos y tres dimensiones
- Cada hilo sabe a que bloque corresponde y cuantos bloques hay en el grid.

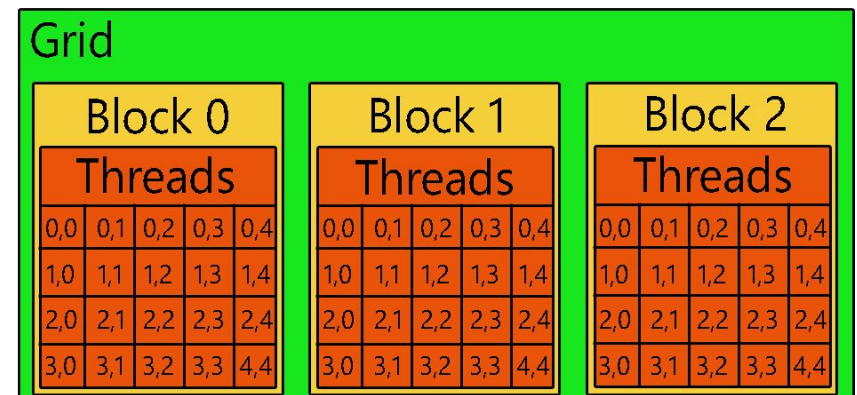
1D Grid of 1D Blocks



1D Grid of 3D Blocks



1D Grid of 2D Blocks



Grids y Blocks

- En CUDA al programar se debe definir cuantos hilos serán usados en cada bloque
- Para eso existen las variables:

blockIdx : Id de bloque dentro de la grilla

threadIdx: Id de hilo dentro del bloque

blockDim: Numero de hilos por bloque

Estas variables tienen un atributo por eje cartesiano x, y, z, por ejemplo para el X

blockIdx.x Id de bloque dentro de la grilla para la dimensión X

threadIdx.x Id de hilos dentro del bloque para la dimensión X

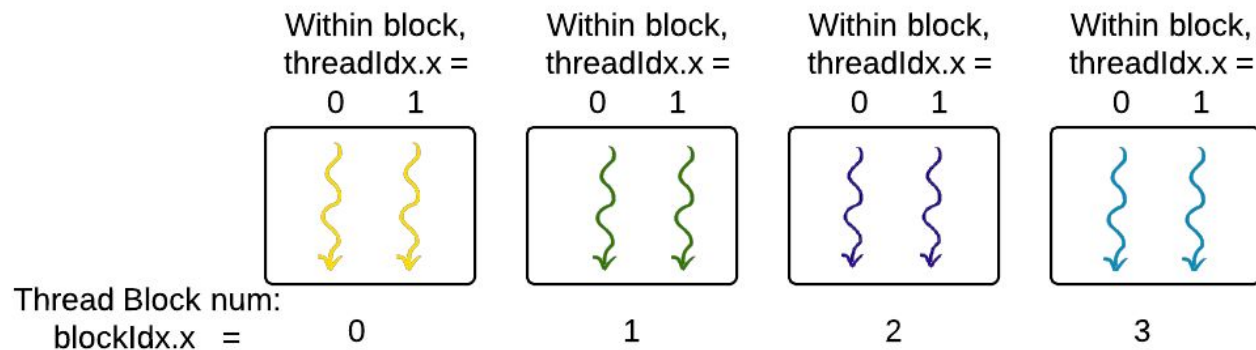
blockDim.x numero de hilos de bloque para la dimensión X

Ejemplo

- Queremos sumar 2 vectores de 8 elementos usando 8 hilos.
- Una posible organización don 4 bloques de dos hilos cada uno.
- Como es un problema de una sola dimensión, usamos la dimensión X
- Para determinar el ID de cada hilos usaremos la ecuación:
- $\text{int tid} = \text{blockDim.x} * \text{blockIdx.x} + \text{threadIdx.x};$

Ejemplo

1- dimensional Grid of Blocks of Threads,
where number of threads per block,
 $\text{blockDim.x} = 2$



Processing Unit Assignment

	0		1		2		3	
index	0	1	2	3	4	5	6	7
Array A								
B								
C								



Practica

- Suma de vectores usando CPU
- Suma de vectores usando un hilo de GPU
- Suma de vectores usando múltiples hilos de GPU

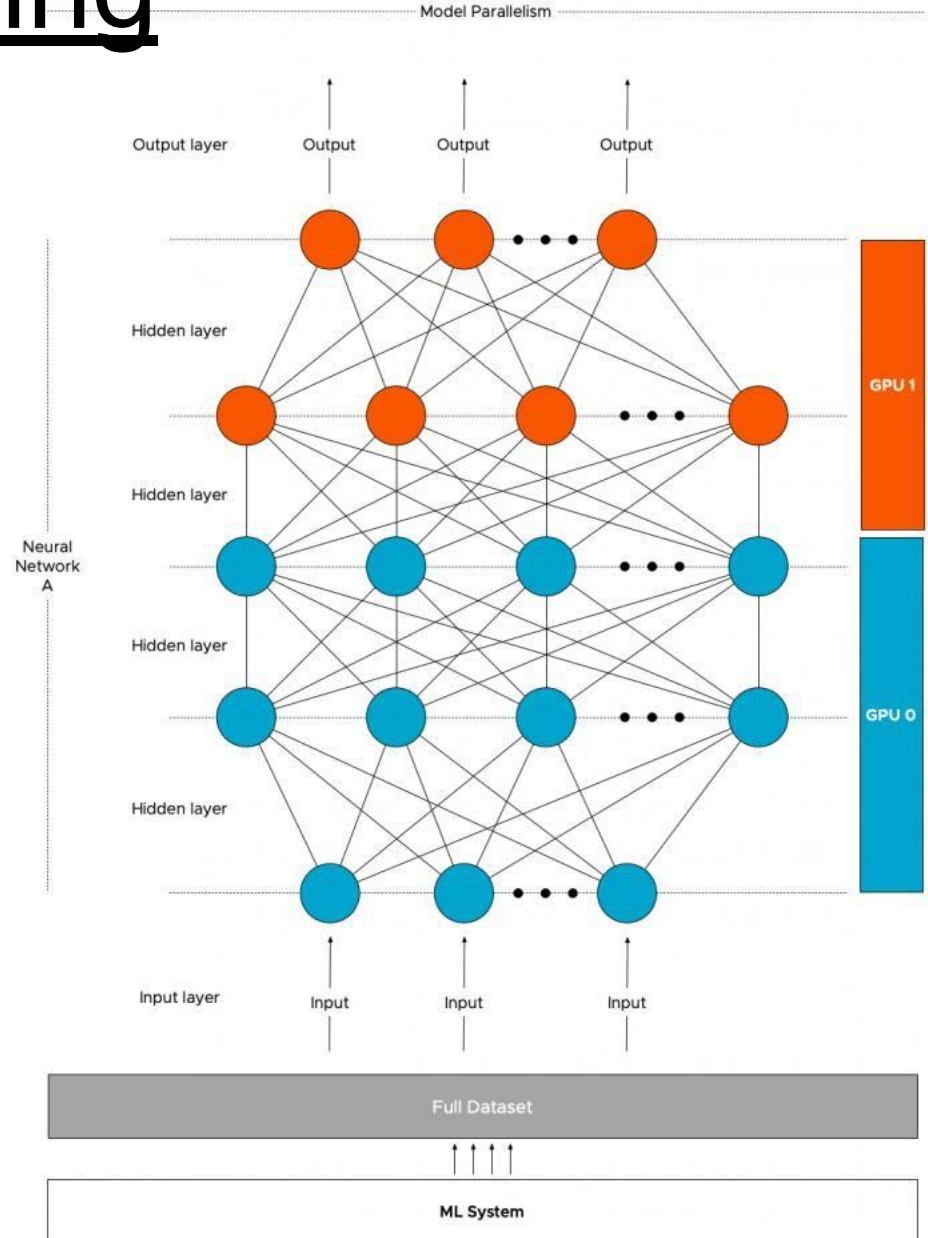


Usos de GPU

- Gráficos
- Machine Learning
- Minería de Criptomonedas.
- Password Cracking

Machine Learning

En **redes neuronales** los nodos pueden realizar su tarea computacional de manera paralela.



Ejemplo Titan XP

- Comando deviceQuery

(./cudasamples/samples/bin/x86_64/linux/release/deviceQuery)



- ✓ 12 GB de Memoria
- ✓ 30 Multiprocesadores
- ✓ 128 CUDA cores por Multiprocesador (= 3840 CUDA Cores)
- ✓ 3 MB de Caché L2
- ✓ 2048 threads máximo por Multiprocesador

Cuda Core != Core



Hash rate

Termino tomado de minería de criptomonedas

La unidad es hash/segundo

Mide cuantos hashes, según el algoritmo puede generar la placa.

Ejemplo Hashcat en Titan

```
svalles@titan-arq:~$ hashcat -b  
hashcat (v4.0.1) starting in benchmark mode...
```

```
OpenCL Platform #1: NVIDIA Corporation  
=====
```

* Device #1: TITAN Xp, 3048/12194 MB allocatable, 30MCU

```
Benchmark relevant options:  
=====
```

* --optimized-kernel-enable

```
Hashmode: 900 - MD4  
  
Speed.Dev.#1.....: 52249.2 MH/s (20.47ms)
```

```
Hashmode: 0 - MD5
```

```
Speed.Dev.#1.....: 31457.3 MH/s (31.99ms)
```

```
Hashmode: 100 - SHA1
```

```
Speed.Dev.#1.....: 11864.4 MH/s (85.33ms)
```

Ejemplo Hashcat en i7 con GeForce

```
CUDA API (CUDA 11.1)
=====
* Device #1: GeForce 940MX, 1689/2048 MB, 3MCU

OpenCL API (OpenCL 1.2 CUDA 11.1.114) - Platform #1 [NVIDIA Corporation]
=====
* Device #2: GeForce 940MX, skipped

OpenCL API (OpenCL 2.1 ) - Platform #2 [Intel(R) Corporation]
=====
* Device #3: Intel(R) HD Graphics 620, 6260/6324 MB (2047 MB allocatable), 24MCU
* Device #4: Intel(R) Core(TM) i7-7500U CPU @ 2.70GHz, skipped

Benchmark relevant options:
=====
* --optimized-kernel-enable

Hashmode: 0 - MD5

Speed.#1.....: 2350.7 MH/s (85.07ms) @ Accel:64 Loops:1024 Thr:1024 Vec:8
Speed.#3.....: 507.6 MH/s (48.06ms) @ Accel:512 Loops:256 Thr:8 Vec:4
Speed.#*.....: 2858.3 MH/s

Hashmode: 100 - SHA1

Speed.#1.....: 782.0 MH/s (63.74ms) @ Accel:16 Loops:1024 Thr:1024 Vec:1
Speed.#3.....: 61136.0 kH/s (48.85ms) @ Accel:32 Loops:512 Thr:8 Vec:4
Speed.#*.....: 843.2 MH/s

Hashmode: 1400 - SHA2-256

Speed.#1.....: 279.7 MH/s (89.49ms) @ Accel:8 Loops:1024 Thr:1024 Vec:1
Speed.#3.....: 67488.3 kH/s (92.23ms) @ Accel:512 Loops:64 Thr:8 Vec:4
Speed.#*.....: 347.2 MH/s

Hashmode: 1700 - SHA2-512
```

Resultados en MH/s

Placa	MD5	SHA1	SHA256
Titan XP	31457	11864	4407
GeForce 940MX	2350	782	279
HD Graphics 620	507	61	64
i7-7500U	226	132	53

